



D.M.G SYSTEM

Local network medical records management software: Python, ttkbootstrap, Flask, PostgreSQL, printable HTML, PyInstaller.

Table of Contents

1. General Project Overview	2
2. Folder Architecture	2

1. General Project Overview

D.M.G SYSTEM is a local medical desktop application. It is designed for a local network: one PC acts as the server, while the other PC's run the client.

The client never accesses PostgreSQL directly. It communicates with the Flask server via HTTP. The server controls authentication, permissions, patients, photos, consultations, notes, and prescriptions.

- Client interface: Python + ttkbootstrap for windows, forms, and the patient table.
- Local server: Flask to expose an HTTP API accessible on the local network.
- Database: PostgreSQL to store users, patients, consultations, notes, prescriptions, and logs.
- Photos: Files saved on the server side in patient_photos; PostgreSQL only keeps the file name.
- Prescriptions: Data saved in the database, then printed via a temporary HTML page opened in the browser.
- Packaging: PyInstaller produces DMG_Server.exe and DMG_Client.exe.

Why this architecture?

Choice	Technical Reason	Advantage
Separate client	The client remains a simple graphical interface.	Multiple PCs can connect to the same server
Flask server	Centralizes the /api routes and permissions.	Prevents clients from manipulating PostgreSQL directly.
PostgreSQL	Solid relational database.	Persistent data, relationships, constraints, cascading deletions.
api_client.py	A single class to talk to the server.	The interface is not cluttered with HTTP details everywhere.
ui_theme.py	Centralizes logo, styles, and PyInstaller paths.	Changing a logo or a color becomes much simpler.

2. Folder Architecture & Folder Roles

Directory Structure:

```
DMG_SYSTEM/  
  assets/images/  
    logo_DMG.png  
    logo_DMG.ico  
  client/  
    main_client.py  
    login_ui.py  
    dashboard_ui.py  
    patient_ui.py  
    api_client.py  
    ui_theme.py  
  server/  
    main_server.py  
    server_ui.py  
    config.py  
    change_admin_password.py  
    api/  
      auth.py  
      patients.py  
      users.py  
      consultations.py  
    database/  
      connection.py  
      schema.sql  
    patient_photos/  
  installer/  
    requirements.txt  
    build_client.bat  
    build_server_ui.bat  
    build_server_console.bat  
    allow_firewall_5000_admin.bat
```

Folder/File Roles:

Folder/File	Role
assets/images	Contains the PNG logo and the ICO icon used by the client, the server, and for printing.
client/main_client.py	Entry point of the client. Creates the main window then displays login or Dashboard.
client/login_ui.py	Login screen: server address, user, password, server test.
client/dashboard_ui.py	Main screen: patients, record, photo, consultations, notes, prescriptions, printing.
client/patient_ui.py	Patient creation form with photo, date, BMI, medical information.
client/api_client.py	Class that sends HTTP requests to the Flask server.
client/ui_theme.py	Styles, logo, PyInstaller compatible paths, images.
server/main_server.py	Creates Flask, registers blueprints, launches the server on 0.0.0.0.
server/server_ui.py	Server interface: start server, manage users, display IP.
server/config.py	PostgreSQL configuration, port, runtime paths, and resources.
server/database/connection.py	DB creation, connections, cursors, migrations, users.
server/database/schema.sql	Main SQL tables.
server/api/auth.py	Login, token, route protection, logs.
server/api/patients.py	Patient routes, photos, consultations, notes, prescriptions.